CLUBE DE ESTUDOS
BY ELEMAR JR

C#
DO JEITO CERTO

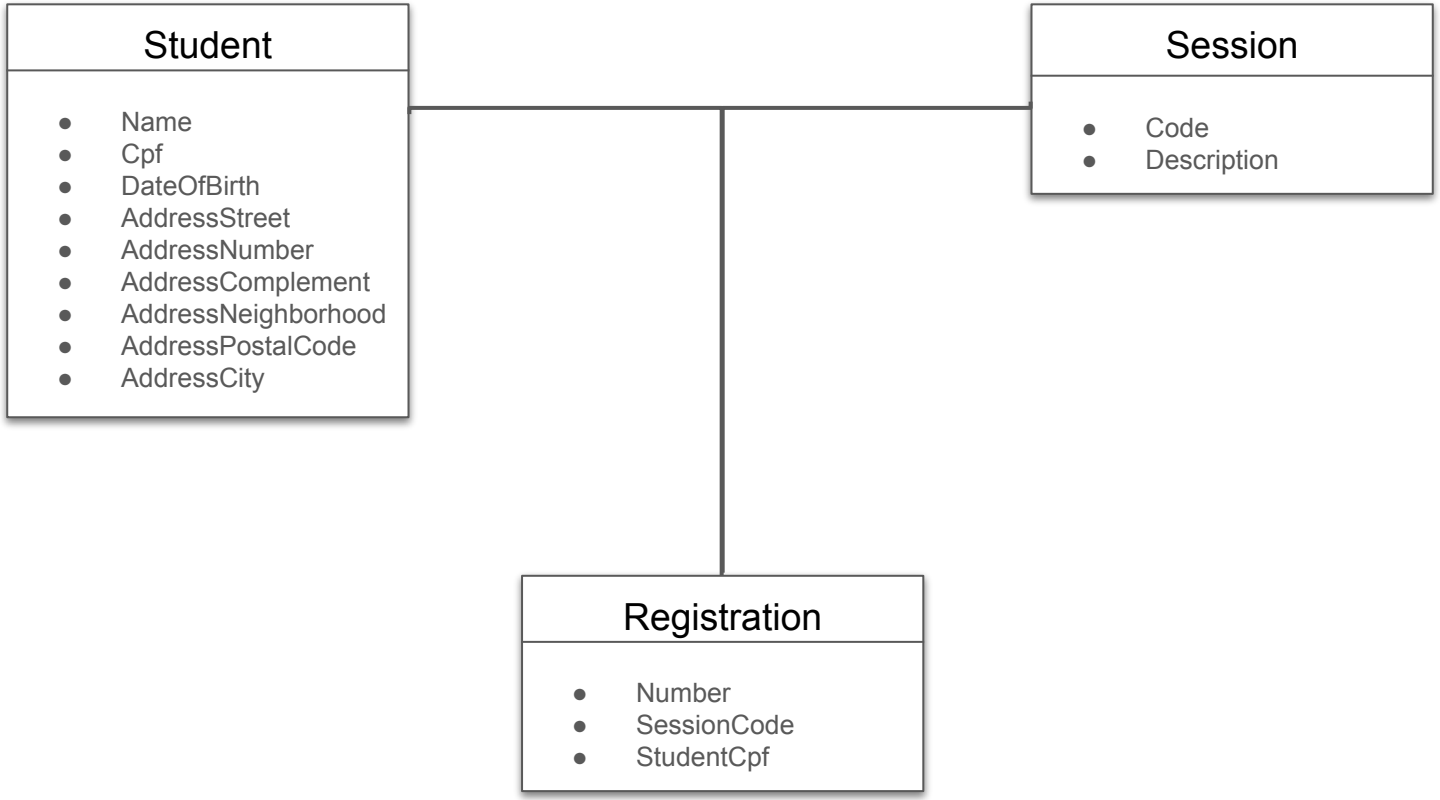
Elemar

AULA 2

ORIENTAÇÃO A OBJETOS
ALÉM DA TEORIA

Elemar

CONSEGUIMOS ENTENDER ESSE SISTEMA?



CLUBE DE ESTUDOS
BY ELEMAR JR

UMA ABORDAGEM INICIAL

```
public class Student
{
    public string Name { get; set; }
    public string Cpf { get; set; }
    public DateOnly DateOfBirth { get; set; }
    public string AddressStreet { get; set; }
    public string AddressNumber { get; set; }
    public string AddressComplement { get; set; }
    public string AddressNeighborhood { get; set; }
    public string AddressPostalCode { get; set; }
    public string AddressCity { get; set; }
}
```

```
public class Session
{
    public string Code { get; set; }
    public string Description { get; set; }
}
```

```
public class Registration
{
    public string Number { get; set; }
    public string SessionCode { get; set; }
    public string StudentCpf { get; set; }
    public DateTime RegisteredIn { get; set; }
}
```

CLUBE DE ESTUDOS
BY ELEMAR JR

QUAL O PROBLEMA ?

```
Session session = new()
{
    Code = "01M",
    Description = "Turma de futebol"
};

Student student = new()
{
    Name = "Júlio Castro Correia",
    Cpf = "84298736431",
    DateOfBirth = new DateOnly(1996, 11, 11),
    AddressStreet = "Rua das Figueiras",
    AddressNumber = "28",
    AddressNeighborhood = "Floresta",
    AddressPostalCode = "93700-000",
    AddressCity = "Campo Bom"
};

Registration registration = new()
{
    SessionCode = session.Code,
    StudentCpf = student.Cpf,
    RegisteredIn = DateTime.Now
};

RegistrationDataAccess registrationDataAccess = new();
registrationDataAccess.Add(registration);
```

CLUBE DE ESTUDOS
BY ELEMAR JR

QUAL O PROBLEMA ?

```
Registration registration = new()
{
    SessionCode = session.Code,
    StudentCpf = student.Cpf,
    RegisteredIn = DateTime.Now
};
```

UM OBJETO **ABERTO** E **INSEGURO**

```
registration.StudentCpf = "";
registration.RegisteredIn = DateTime.Now.AddHours(5);
```

EM UM DESIGN "MAIS SIMPLES" CORREMOS O RISCO DE **COLOCAR OBJETOS EM ESTADO INVÁLIDO**

CAPÍTULO 1

ENCAPSULAMENTO NA PRÁTICA

Elemar

ENCAPSULAMENTO NA PRÁTICA

```
public class Person
{
    public string Name { get; set; }
}
```

```
public class Person
{
    private string _name;

    public string GetName()
        => _name;

    public void SetName(string value)
    {
        _name = value;
    }
}
```

ENCAPSULAMENTO NA PRÁTICA

```
public class Registration
{
    public Registration(string number, string sessionCode, string studentCpf)
    {
        Number = number;
        SessionCode = sessionCode;
        StudentCpf = IsCpf(studentCpf)
           ? studentCpf
           : throw new ArgumentException("Invalid CPF.", nameof(studentCpf));
        RegisteredIn = DateTime.Now;
    }

    public string Number { get; }
    public string SessionCode { get; private set; }
    public string StudentCpf { get; }
    public DateTime RegisteredIn { get; }

    public void ChangeSession(string sessionCode)
    {
        SessionCode = sessionCode;
        // More changes
    }

}
```

O código da turma pode ser alterado internamente

ENCAPSULAMENTO NA PRÁTICA

```
public class Registration
{
    public Registration(string number, string classCode, string studentCpf)
    {
        Number = number;
        ClassCode = classCode;
        StudentCpf = IsCpf(studentCpf)
           ? studentCpf
           : throw new ArgumentException("Invalid CPF.", nameof(studentCpf));
        RegisteredIn = DateTime.Now;
    }

    public string Number { get; }
    public string ClassCode { get; private set; }
    public string StudentCpf { get; }
    public DateTime RegisteredIn { get; }

    public void ChangeClass(string classCode)
    {
        ClassCode = classCode;
        // More changes
    }

    public bool IsCpf(string cpf) { /* CPF validation */ }
}
```

Objeto não é criado em um estado inválido

Validação dos dados

OBJETOS NÃO DEVEM REVELAR TODOS AS SUAS **PROPRIEDADES** E **MÉTODOS**

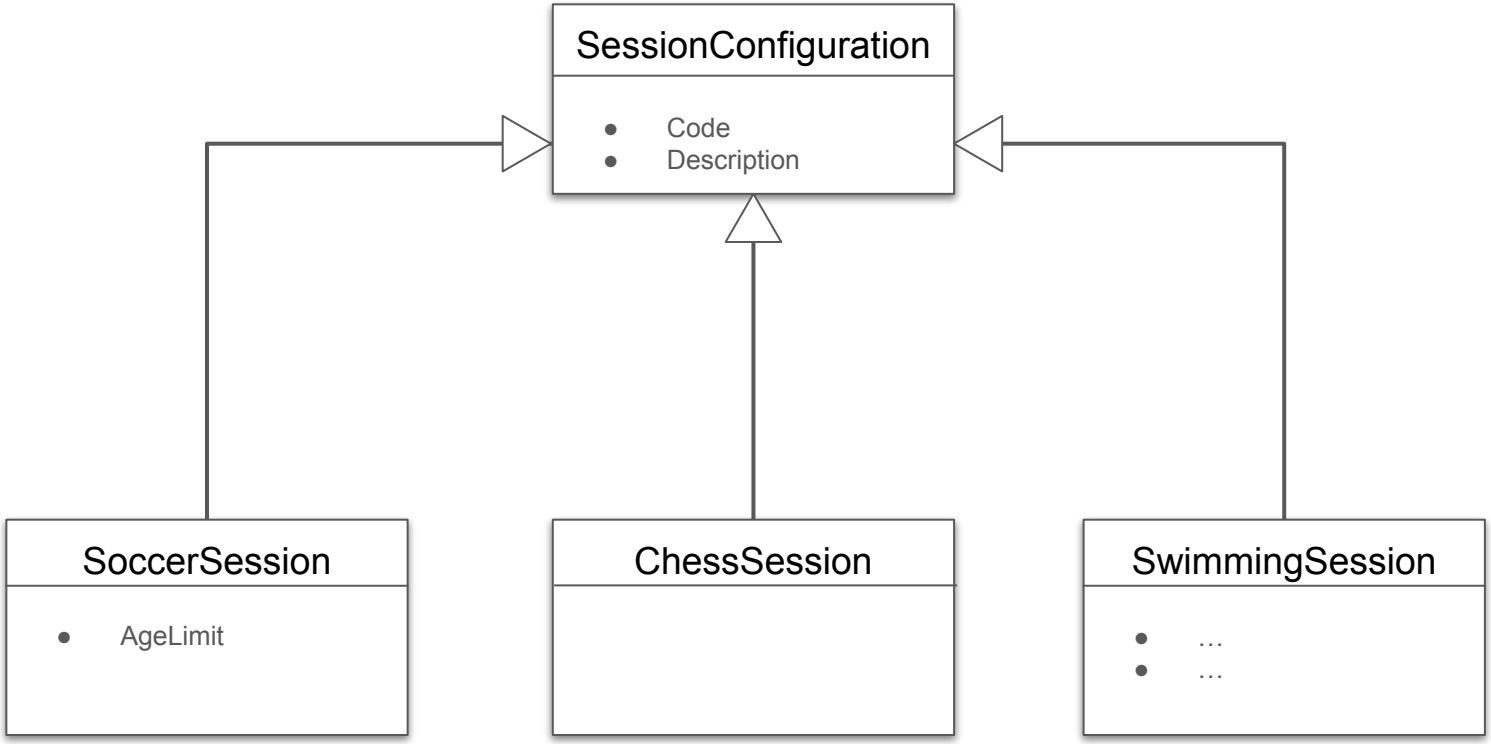
O CONJUNTO DE **PROPRIEDADES** E **MÉTODOS PÚBLICOS** DE UM OBJETO **CONSTITUEM** A SUA "**INTERFACE**"

CAPÍTULO 2

HERANÇA NA PRÁTICA

Elemar

HERANÇA NA PRÁTICA



HERANÇA NA PRÁTICA

Classe base. Não pode ser instanciada

```
public abstract class ClassConfiguration
{
    protected ClassConfiguration(string code, string description)
    {
        Code = code;
        Description = description;
    }

    public string Code { get; }
    public string Description { get; }

    public abstract bool CanRegister(Student student);

    protected virtual bool IsOpen()
        => true;
}
```

Acessível apenas dentro da própria classe e das classes derivadas

Requer que as classes derivadas implementem esse método

Permite que esse método seja sobrescrito pelas classes derivadas

A CLASSE QUE HERDAR DEVERÁ FICAR ASSIM

```
public class SoccerClass : ClassConfiguration
{
    public SoccerClass(string code, string description, int ageLimit)
        : base(code, description)
    {
        AgeLimit = ageLimit;
    }

    public int AgeLimit { get; }

    public override bool CanRegister(Student student)
        => IsOpen() && student.Age <= AgeLimit;

    protected override bool IsOpen()
        => false;
}
```

A CLASSE QUE HERDAR DEVERÁ FICAR ASSIM

```
Student student = new(
    name: "Júlio Castro Correia",
    cpf: "84298736431",
    dateOfBirth: new DateOnly(1996, 11, 11),
    addressStreet: "Rua das Figueiras",
    addressNumber: "28",
    addressNeighborhood: "Floresta",
    addressPostalCode: "93700-000",
    addressCity: "Campo Bom");

SoccerClass soccerClass = new("01F", "Turma de Futebol A", 17);
if (!soccerClass.CanRegister(student))
    throw new Exception("Aluno não pode realizar inscrição na turma.");

Registration registration = new("12345", soccerClass.Code, student.Cpf);

RegistrationDataAccess registrationDataAccess = new();
registrationDataAccess.Add(registration);
```

DEFINIÇÕES DE HERANÇA

É UMA ESTRATÉGIA DE **EXTENSÃO**

A CLASSE CUJOS MEMBROS SÃO **HERDADOS** É CONHECIDA COMO **CLASSE BASE**

AS CLASSES QUE **HERDAM** ESSES MEMBROS SÃO CHAMADAS DE **CLASSES DERIVADAS**

IS-A É UMA FORMA PARA IDENTIFICARMOS SE FAZ SENTIDO A UTILIZAÇÃO DE **HERANÇA**. ELA CONSISTE EM ESTABELECER UMA RELAÇÃO "**É UM**" ENTRE UMA **CLASSE DERIVADA** E **UMA CLASSE BASE**

CAPÍTULO 3

POLIMORFISMO NA PRÁTICA

Elemar

POLIMORFISMO

PRINCÍPIO ONDE **CLASSES DERIVADAS** INVOCAM MÉTODOS QUE, EMBORA TENHAM A MESMA ASSINATURA, POSSUEM UM **COMPORTAMENTO EXCLUSIVO** DEFINIDO POR CADA CLASSE

ISSO OCORRE POIS AS **CLASSES DERIVADAS** **HERDAM** A **INTERFACE PÚBLICA** DE UMA CLASSE BASE

POLIMORFISMO

```
public class ValidateRegistrationService
{
    public IEnumerable<ClassConfiguration> Validate()
    {
        Student student = new(
           name: "Júlio Castro Correia",
           cpf: "84298736431",
           dateOfBirth: new DateOnly(1996, 11, 11),
           addressStreet: "Rua das Figueiras",
           addressNumber: "28",
           addressNeighborhood: "Floresta",
           addressPostalCode: "93700-000",
           addressCity: "Campo Bom");

        var classConfigurationDataAccess = new ClassConfigurationDataAccess();
        IEnumerable<ClassConfiguration> configurations = classConfigurationDataAccess.GetConfigurations();
        foreach (var configuration in configurations)
        {
            if (!configuration.CanRegister(student))
                continue;

            yield return configuration;
        }
    }
}
```

O objeto instanciado é do tipo de uma das classes derivadas

POLIMORFISMO

```
var classConfigurationDataAccess = new ClassConfigurationDataAccess();
IEnumerable<ClassConfiguration> configurations = classConfigurationDataAccess.GetConfigurations();
foreach (var configuration in configurations)
{
    if (!configuration.CanRegister(student))
        continue;
    yield return configuration;
}
```

CAPÍTULO 4

COMPOSIÇÃO NA PRÁTICA

Elemar

OBSESSÃO POR TIPOS PRIMITIVOS

SÃO TIPOS COMO **STRING**, **LONG**, **DOUBLE**, **INT**, **BOOL**, ETC

DADOS EM UM LOCAL E **COMPORTAMENTO** EM OUTRO

NO **ENCAPSULAMENTO** VIMOS QUE OS DADOS E O COMPORTAMENTO **NÃO DEVEM ESTAR SEPARADOS**, MAS **COMBINADOS EM OBJETOS**

A SOLUÇÃO PARA ESSE PROBLEMA RESIDE NA ADOÇÃO DE PRÁTICAS DE OO

COMPOSIÇÃO

```
public class Student
{
    public Student(string name, string cpf, DateOnly dateOfBirth, string addressStreet, string addressNumber,
        string addressNeighborhood, string addressPostalCode, string addressCity, string addressComplement = "")
    {
        Name = name;
        Cpf = cpf;
        DateOfBirth = dateOfBirth;
        AddressStreet = addressStreet;
        AddressNumber = addressNumber;
        AddressComplement = addressComplement;
        AddressNeighborhood = addressNeighborhood;
        AddressPostalCode = addressPostalCode;
        AddressCity = addressCity;
    }

    public string Name { get; }
    public string Cpf { get; }
    public DateOnly DateOfBirth { get; }
    public string AddressStreet { get; }
    public string AddressNumber { get; }
    public string AddressComplement { get; }
    public string AddressNeighborhood { get; }
    public string AddressPostalCode { get; }
    public string AddressCity { get; }
    public int Age => CalculateAge(DateOfBirth);

    private int CalculateAge(DateOnly dateOfBirth) { /* Age calculation */ }
}
```

COMPOSIÇÃO

```
public class Student
{
    public Student(string name, string cpf, DateOnly dateOfBirth, Address address)
    {
        Name = name;
        Cpf = cpf;
        DateOfBirth = dateOfBirth;
        Address = address;
    }

    public string Name { get; }
    public string Cpf { get; }
    public DateOnly DateOfBirth { get; }
    public Address Address { get; }
    public int Age => CalculateAge(DateOfBirth);

}
```

```
public readonly record struct Address(
    string Street,
    string Number,
    string Neighborhood,
    string PostalCode,
    string City,
    string Complement = "");
```

CONTINUA SENDO UMA **STRUCT** COM TODOS OS SEUS **BENEFÍCIOS**

BÔNUS DO **SUGAR SINTAX** PROVIDO PELO **RECORD**

ADDRESS É UM EXEMPLO DO QUE COMUMENTE CHAMAMOS DE **VALUE OBJECT**

CPF É OUTRO CASO QUE PODEMOS MELHORAR CRIANDO UM TIPO MAIS EXPRESSIVO DO QUE A *STRING*

```
public class Student
{
    public Student(string name, string cpf, DateOnly dateOfBirth, Address address)
    {
        Name = name;
        Cpf = cpf;
        DateOfBirth = dateOfBirth;
        Address = address;
    }

    public string Name { get; }
    public string Cpf { get; }
    public DateOnly DateOfBirth { get; }
    public Address Address { get; }
    public int Age => CalculateAge(DateOfBirth);

    private int CalculateAge(DateOnly dateOfBirth) { /* Age calculation */ }
}
```

COMPOSIÇÃO

```
public struct Cpf
{
    private readonly string _value;

    public Cpf(string value) => _value = value;

    public static Cpf Parse(string value)
       => (TryParse(value, out var result))
         ? result
         : throw new ArgumentException(nameof(value));

    public static bool TryParse(string value, out Cpf result)
    {
        if (string.IsNullOrEmpty(value) || !IsValid(value))
        {
            result = default;
            return false;
        }

        result = new Cpf(value);
        return true;
    }

    public static implicit operator Cpf(string value) => Parse(value);
    public override string ToString() => _value;
    public bool IsEmpty => (_value != null);

    public static bool IsValid(string value) { /* CPF validation */ }
}
```

COMPOSIÇÃO

```
public struct Cpf
{
    private readonly string _value;

    public Cpf(string value) => _value = value;

    public static Cpf Parse(string value)
       => (TryParse(value, out var result))
         ? result
         : throw new ArgumentException(nameof(value));

    public static bool TryParse(string value, out Cpf result)
    {
        if (string.IsNullOrEmpty(value) || !IsValid(value))
        {
            result = default;
            return false;
        }

        result = new Cpf(value);
        return true;
    }

    public static implicit operator Cpf(string value) => Parse(value);
    public override string ToString() => _value;
    public bool IsEmpty => (_value != null);

    public static bool IsValid(string value) { /* CPF validation */ }
}
```

Encapsulamento em uso. O atributo é invisível para qualquer objeto externo.

Uma vez atribuído um valor, ele não pode mais ser modificado

Operadores implícitos para tipos primitivos facilitam a atualização da base de código

O acesso ao valor ocorre através do método *ToString()*

COMPOSIÇÃO

```
public struct Cpf
{
    private readonly string _value;

    public Cpf(string value) => _value = value;

    public static Cpf Parse(string value)
       => (TryParse(value, out var result))
         ? result
         : throw new ArgumentException(nameof(value));

    public static bool TryParse(string value, out Cpf result)
    {
        if (string.IsNullOrEmpty(value) || !IsValid(value))
        {
            result = default;
            return false;
        }

        result = new Cpf(value);
        return true;
    }

    public static implicit operator Cpf(string value) => Parse(value);
    public override string ToString() => _value;
    public bool IsEmpty => (_value != null);

    public static bool IsValid(string value) { /* CPF validation */ }
}
```

Ao criarmos classes para encapsular dados e comportamentos é interessante seguirmos padrões vigentes

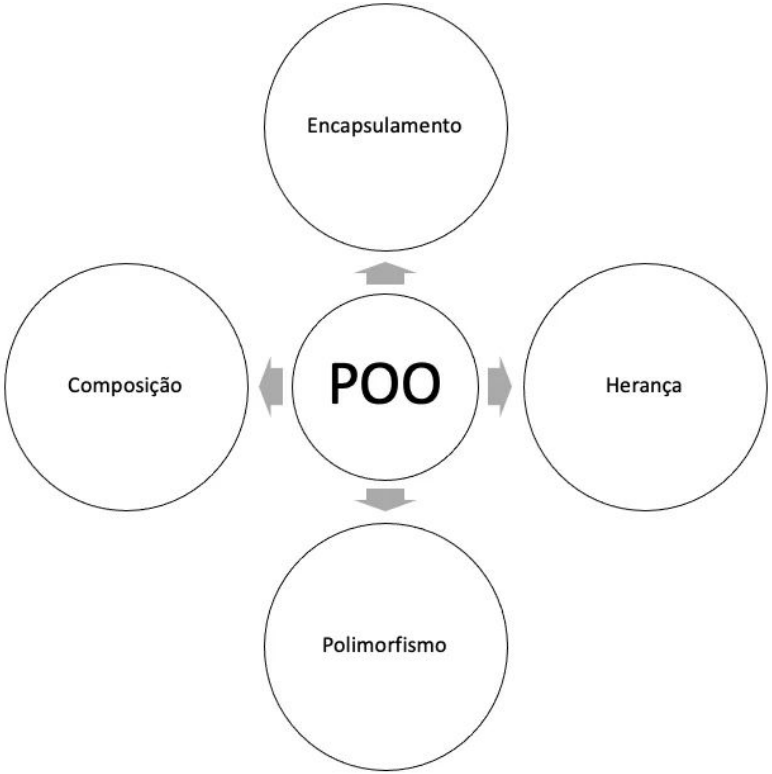
COMPOSIÇÃO

PRINCÍPIO QUE DEFINE QUE **UM OBJETO PODE CONTER OUTRO OBJETO** (COMO UM CARRO QUE POSSUI UM MOTOR)

O OBJETO CONTIDO PODE SER UMA **CLASSE**, **RECORD** OU **STRUCT**

HAS-A É UMA FORMA PARA **IDENTIFICARMOS** SE FAZ SENTIDO A UTILIZAÇÃO DE **COMPOSIÇÃO**. ELA CONSISTE EM ESTABELECER UMA **RELAÇÃO "TEM UM" ENTRE DUAS CLASSES**

FERRAMENTAS FUNDAMENTAIS DE OO



CLUBE DE ESTUDOS
BY ELEMAR JR

CAPÍTULO 5

GENERICS NO DIA-A-DIA

Elemar

GENERICS NOS PROPORCIONAM UM CÓDIGO **PERFORMÁTICO** E **REUTILIZÁVEL**

CLUBE DE ESTUDOS
BY ELEMAR JR

EXEMPLOS DE UTILIZAÇÃO NO FRAMEWORK

```
var lista = new List<int>(capacity: 10);
var tupla = new Tuple<double, string>(1.5, "Tupla");
var dicionario = new Dictionary<Guid, Person>();

Action<string> imprimir = (mensagem) => Console.WriteLine($"Log: {mensagem}");
imprimir("C# do Jeito Certo");

Func<int, bool> ehPar = (numero) => numero % 2 == 0;
Console.WriteLine(ehPar(2));
```

TRABALHANDO COM RESTRIÇÕES

Aplica as restrições ao parâmetro

```
public class PromocaoRelampago<T> where T : IItemVendavel
{
    public void AplicarDesconto(T item)
    {
        Console.WriteLine($"Novo preço do {item.Nome}: R$ {item.Preco}");
    }
}
```

TRABALHANDO COM RESTRIÇÕES

```
public class Organization<TEmployee, TWork>
    where TEmployee : Employee, new()
    where TWork : IWork
{
    // Código da classe
}
```

Obriga que a classe tenha um construtor vazio

TRABALHANDO COM RESTRIÇÕES

```
public class Organization<TEmployee, TWork>
    where TEmployee : notnull
    where TWork : IWork
{
    // Código da classe
}
```

Obriga que o tipo informado não seja *nullable*

TRABALHANDO COM RESTRIÇÕES

```
public class UtilitarioEstoque
{
    // TOrigem: O tipo específico (Ex: Notebook)
    // TDestino: O tipo genérico da lista principal (Ex: Produto)

    // A Constraint "where TOrigem : TDestino" garante o polimorfismo
    // Ela diz: "Só aceito copiar se TOrigem for filho de TDestino"
    public void MoverItens<TOrigem, TDestino>(
        List<TOrigem> itensNovos,
        List<TDestino> estoqueGeral)
        where TOrigem : TDestino
    {
        foreach (var item in itensNovos)
        {
            // O compilador só permite essa linha
            // porque garantiu que TOrigem HERDA de TDestino.
            estoqueGeral.Add(item);
        }

        Console.WriteLine($"{itensNovos.Count} itens movidos com sucesso.");
    }
}
```

COVARIÂNCIA E CONTRAVARIÂNCIA

```
public class Employee { }

public class Manager : Employee { }


void ManagerPrinter(List<Employee> employees)
{
    // Executa o método
}


var managers = new List<Manager>
{
    new Manager { Name = "Alice", Department = "HR" },
    new Manager { Name = "Bob", Department = "IT" }
};

ManagerPrinter(managers);
```

COVARIÂNCIA E CONTRAVARIÂNCIA

```
Argument type 'System.Collections.Generic.List<Manager>' is not assignable to parameter type 'System.Collections.Generic.List<Employee>'

local variable List<Manager> managers
```

COVARIÂNCIA E CONTRAVARIÂNCIA

```
void ManagerPrinter(IEnumerable<Employee> employees)
{
    // Executa o método
}
```

PERMITE A UTILIZAÇÃO DE UM TIPO MAIS DERIVADO DO QUE O ORIGINAL

COVARIÂNCIA E CONTRAVARIÂNCIA

```
// 1. O Método Genérico (Sabe lidar com QUALQUER Employee)
// Pense nisso como um "Clínico Geral"
void ProcessarFuncionario(Employee e)
{
    Console.WriteLine($"Processando: {e.Name}");
}

// 2. O Erro Intuitivo vs A Realidade
// Imagine que uma variável pede uma ação EXCLUSIVA para Gerentes
Action<Manager> acaoParaGerente;

// CONTRAVARIÂNCIA EM AÇÃO:
// Eu posso atribuir o método que lida com Funcionário (Geral)
// para a variável que espera lidar com Gerente (Específico)
acaoParaGerente = ProcessarFuncionario;


// Por que funciona?
// Quando eu chamar acaoParaGerente(umGerente),
// o método ProcessarFuncionario vai receber esse gerente
// Como Gerente É um Funcionário, o método funciona perfeitamente
acaoParaGerente(new Manager { Name = "Bob" });
```

GENERICS EM MÉTODOS

```
public class Utilitarios
{
    // OBSERVE: A classe "Utilitarios" NÃO é genérica.
    // Apenas este método é. O <T> vive apenas aqui dentro.
    public static void TrocarValores<T>(ref T a, ref T b)
    {
        T temporario = a;
        a = b;
        b = temporario;
    }
}
```

CAPÍTULO 5

TRABALHANDO FALHAS NA PRÁTICA

Elemar

FALHAS NA PRATICA

EXCEPTIONS SÃO, DE FATO, RUINS ?

EXCEPTIONS

C# **NATIVAMENTE** TRATA **FALHAS** À PARTIR DE **EXCEÇÕES**

É A FORMA QUE AS **BIBLIOTECAS NATIVAS** DO C# TRATAM **FALHAS**

TRATAR FALHAS COM **EXCEPTIONS** É MAIS **FAMILIAR** PARA **DESENVOLVEDORES** C# DO QUE ABORDAGENS **FUNCIONAIS**

TRATANDO FALHAS NA PRÁTICA

```
int.Parse("Eximia");
```



```
public static int Parse(string s, NumberStyles style, IFormatProvider? provider)
{
    if (s is null) { ThrowHelper.ThrowArgumentNullException(ExceptionArgument.s); }
    return Parse(s.AsSpan(), style, provider);
}
```

TRATANDO FALHAS NA PRÁTICA

```
File.ReadAllText(@"C:\Users\Eximia\Teste.txt");
```

```
public static string ReadAllText(string path, Encoding encoding)
{
    Validate(path, encoding);

    using StreamReader sr = new StreamReader(path, encoding, detectEncodingFromByteOrderMarks: true);
    return sr.ReadToEnd();
}
```

```
private static void Validate(string path, Encoding encoding)
{
    ArgumentException.ThrowIfNullOrEmpty(path);
    ArgumentNullException.ThrowIfNull(encoding);
}
```

FALHAS NA PRÁTICA

TRABALHANDO COM EXCEPTIONS DO JEITO CERTO

TRATAR CONDIÇÕES COMUNS

CONTORNE **EXCEÇÕES** COM **BOA PROBABILIDADE** DE OCORRER

```
if (conn.State != ConnectionState.Closed)
    conn.Close();
```

AVALIE A UTILIZAÇÃO DOS MÉTODOS TRY

MUITAS VEZES, OPTAR POR **OUTROS MÉTODOS** QUE N**ÃO GERAM EXCEÇÕES** É UMA BOA IDEIA, COMO O **INT.TRYPARSE**, POR EXEMPLO

SE É **IMPRESCINDÍVEL** QUE A **CONVERSÃO** DO VALOR OCORRA COM **SUCESSO** PARA O SEGUIMENTO DO **FLUXO**, O **MANTIMENTO** DA POSSÍVEL **EXCEÇÃO** É **RECOMENDÁVEL**

PROJEÇÃO DAS NOSSAS CLASSES

DEVEMOS **PROJETAR** AS NOSSAS CLASSES DE FORMA QUE **EXCEÇÕES** POSSAM SER **EVITADAS**

A CLASSE **FileStream**, POR EXEMPLO, OFERECE FORMAS DE **IDENTIFICARMOS** SE JÁ CHEGAMOS AO **FINAL** DO **ARQUIVO**

```
public static void ReadAll(FileStream fileToRead)
{
    ArgumentNullException.ThrowIfNull(fileToRead);

    int b;
    fileToRead.Seek(0, SeekOrigin.Begin);
    for (int i = 0; i < fileToRead.Length; i++)
    {
        b = fileToRead.ReadByte();
        // Do something with the byte
    }
}
```

CUIDADO AO LANÇAR EXCEÇÕES

```
try
{
    // Code
}
catch (Exception ex)
{
    // Log
    throw new Exception(ex.Message);
}
```

```
try
{
    // Code
}
catch (Exception ex)
{
    // Log
    throw ex;
}
```

CUIDADO AO LANÇAR EXCEÇÕES

UMA DAS COISAS MAIS IMPORTANTES QUE UMA **EXCEÇÃO** CARREGA É A **STACK TRACE**

PRECISAMOS TOMAR O CUIDADO DE **PRESERVÁ-LA**

```
try
{
    // Code
}
catch (Exception ex)
{
    // Log
    throw;
}
```

```
try
{
    // Code
}
catch (Exception ex)
{
    // Log
    throw new Exception("New exception.", ex);
}
```

USAR TIPOS DE EXCEÇÕES EXISTENTES

A **CRIAÇÃO** DE UM **NOVO TIPO DE EXCEÇÃO** DEVE OCORRER APENAS QUANDO OS **TIPOS JÁ EXISTENTES** NÃO FOREM **SUFICIENTES**

NÃO LANCE TIPOS DE EXCEÇÕES **GENÉRICAS**

System.Exception
System.ApplicationException

NÃO LANCE TIPOS DE EXCEÇÕES **RESERVADAS**

System.NullReferenceException
System.OutOfMemoryException
System.StackOverflowException

USAR TIPOS DE EXCEÇÕES EXISTENTES

Argumento inválido	Exceção
Parâmetro null	ArgumentNullException
Fora do intervalo permitido de valores	ArgumentOutOfRangeException
Valor do enum inválido	InvalidEnumArgumentException
Contém um formato que não atende às especificações	FormatException
É inválido	ArgumentException

Siga a diretiva **CA2201**!

PRIORIZE A UTILIZAÇÃO DE **MÉTODOS ESTÁTICOS AUXILIARES** PARA O LANÇAMENTO DE **EXCEÇÕES**

```
ArgumentNullException.ThrowIfNull
ArgumentException.ThrowIfNullOrEmpty(String, String)
ArgumentException.ThrowIfNullOrWhiteSpace(String, String)
ArgumentOutOfRangeException.ThrowIfZero<T>(T, String)
ArgumentOutOfRangeException.ThrowIfNegative<T>(T, String)
ArgumentOutOfRangeException.ThrowIfEqual<T>(T, T, String)
ArgumentOutOfRangeException.ThrowIfLessThan<T>(T, T, String)
ArgumentOutOfRangeException.ThrowIfNotEqual<T>(T, T, String)
ArgumentOutOfRangeException.ThrowIfNegativeOrZero<T>(T, String)
ArgumentOutOfRangeException.ThrowIfGreaterThan<T>(T, T, String)
ArgumentOutOfRangeException.ThrowIfLessThanOrEqual<T>(T, T, String)
ArgumentOutOfRangeException.ThrowIfGreaterThanOrEqual<T>(T, T, String)
ObjectDisposedException.ThrowIf
```

EXCEÇÃO PERSONALIZADA

IMPLEMENTE OS **TRÊS CONSTRUTORES PADRÕES** DE CLASSES DE EXCEÇÃO

IMPLEMENTE UMA **EXCEÇÃO PERSONALIZADA** CASO NENHUMA EXISTENTE ATENDA

CADA NOVA **EXCEÇÃO** DEVE IMPLICAR NUMA **NOVA FORMA DE TRATAMENTO** OU **NOVO TIPO DE PROBLEMA**

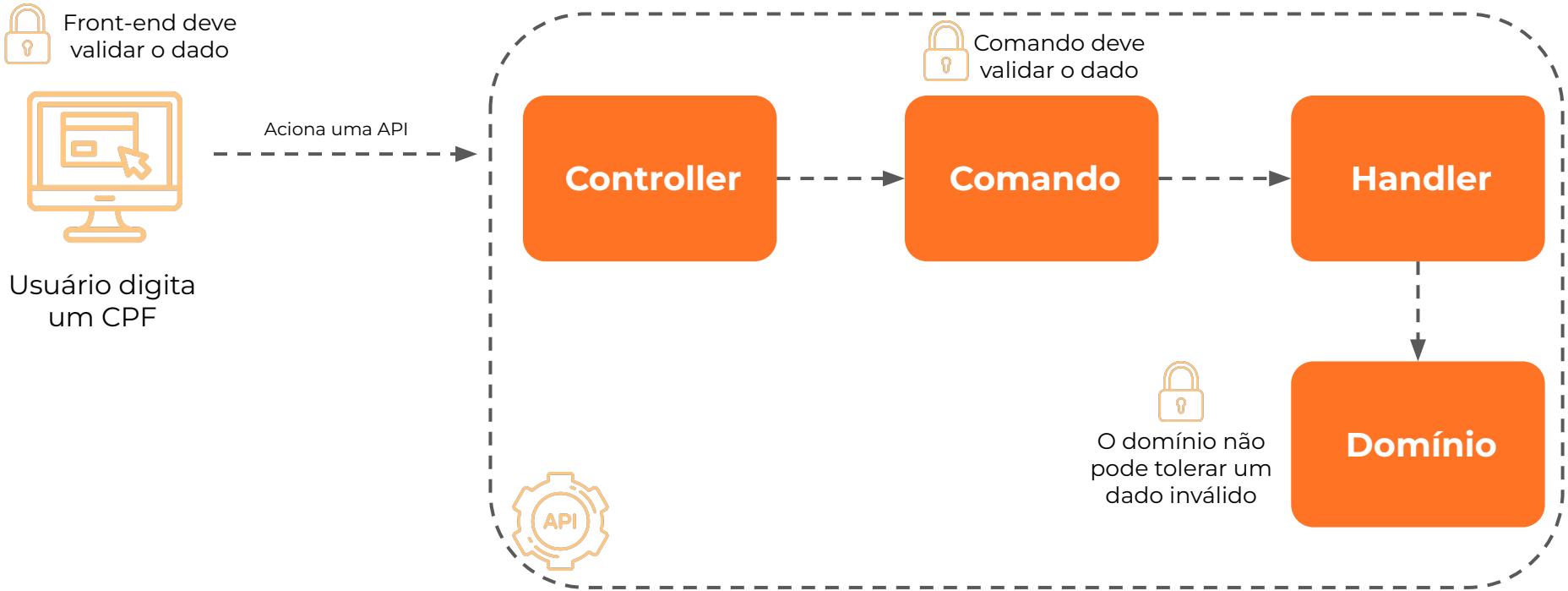
TRABALHANDO COM EXCEPTIONS DO JEITO CERTO

```
public class StudentCannotRegisterInTheClassException : Exception
{
    public string? StudentName { get; }
    public string? ClassCode { get; }

    public StudentCannotRegisterInTheClassException() { }

    public StudentCannotRegisterInTheClassException(string message)
        : base(message) { }

    public StudentCannotRegisterInTheClassException(string message, Exception innerException)
        : base(message, innerException) { }

    public StudentCannotRegisterInTheClassException(string message, string studentName, string classCode)
        : this(message)
    {
        StudentName = studentName;
        ClassCode = classCode;
    }

    [DoesNotReturn]
    public static void ThrowIf(bool condition, string message, string studentName, string classCode)
    {
        if (condition)
            throw new StudentCannotRegisterInTheClassException(message, studentName, classCode);
    }
}
```

TRABALHANDO COM EXCEPTIONS DO JEITO CERTO

EXCEÇÕES DEVEM SER PENSADAS PRIMEIRO PELA **SEMÂNTICA**, DEPOIS PELA **PERFORMANCE**.

O DOMÍNIO NÃO PODE SER SIMPÁTICO!



FALHAS NA PRÁTICA

UTILIZANDO UMA ABORDAGEM ALTERNATIVA

CONSIDERAÇÕES SOBRE A ASSINATURA DESSE MÉTODO

```
public Session GetById(string code)
{ /* Code */ }
```

QUAL SERIA O SEU **RETORNO** CASO NÃO HOUVESSE UMA TURMA COM O CÓDIGO INFORMADO? **NULL**? UMA **EXCEPTION**?

O QUE DEVERIA OCORRER CASO HOUVESSE UM **PROBLEMA DE CONEXÃO COM O DB**? UMA **EXCEPTION**?

O **CLIENTE** DESSE MÉTODO, **SEM ABRÍ-LO**, NÃO TEM COMO SABER SE UMA **EXCEPTION** É LANÇADA NELE OU NÃO

ORIENTAÇÃO A OBJETOS - TRATAMENTO DE FALHAS

```
public readonly struct Result<T> : IResult
{
    private Result(string error, bool isFailure, T value)
    {
        Error = error;
        IsFailure = isFailure;
        Value = value;
    }

    public string Error { get; }
    public bool IsFailure { get; }
    public T Value { get; }
    public bool IsSuccess => !IsFailure;

    public static Result<T> Failure(string error)
        => new Result<T>(error, true, default!);

    public static implicit operator Result<T>(T value)
        => new Result<T>(string.Empty, false, value);

    public static implicit operator Result<T>(Result result)
        => new Result<T>(result.Error, true, default!);
}
```

```
public readonly struct Result : IResult
{
    public Result(string error, bool isFailure)
    {
        Error = error;
        IsFailure = isFailure;
    }

    public string Error { get; }
    public bool IsFailure { get; }
    public bool IsSuccess => !IsFailure;

    public static Result Success()
        => new Result(string.Empty, false);

    public static Result Failure(string error)
        => new Result(error, true);
}
```

ORIENTAÇÃO A OBJETOS - TRATAMENTO DE FALHAS

```
public readonly struct Result<T> : IResult
{
    private Result(string error, bool isFailure, T value)
    {
        Error = error;
        IsFailure = isFailure;
        Value = value;
    }

    public string Error { get; }
    public bool IsFailure { get; }
    public T Value { get; }
    public bool IsSuccess => !IsFailure;

    public static Result<T> Failure(string error)
        => new Result<T>(error, true, default!);

    public static implicit operator Result<T>(T value)
        => new Result<T>(string.Empty, false, value);

    public static implicit operator Result<T>(Result result)
        => new Result<T>(result.Error, true, default!);
}
```

GENERICS:
VIABILIZA A **ABSTRAÇÃO**
DIMINUI A **DUPLICIDADE** DE **CÓDIGO**, POIS A MESMA IMPLEMENTAÇÃO CONCRETA ATUA EM **DIFERENTES CONTEXTOS**

O CLIENTE ESPECIFICA O **TIPO DE OBJETO** QUE DESEJA QUE SEJA **RETORNADO**

ORIENTAÇÃO A OBJETOS - TRATAMENTO DE FALHAS

```
public readonly struct Result<T> : IResult
{
    private Result(string error, bool isFailure, T value)
    {
        Error = error;
        IsFailure = isFailure;
        Value = value;
    }

    public string Error { get; }
    public bool IsFailure { get; }
    public T Value { get; }
    public bool IsSuccess => !IsFailure;

    public static Result<T> Failure(string error)
        => new Result<T>(error, true, default!);

    public static implicit operator Result<T>(T value)
        => new Result<T>(string.Empty, false, value);

    public static implicit operator Result<T>(Result result)
        => new Result<T>(result.Error, true, default!);
}
```

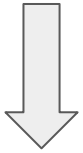
OPERADORES:

REALIZA A **CONVERSÃO** (NESSE CASO **IMPLÍCITA**) DO TIPO T PARA RESULT<T> (PRIMEIRO OPERADOR) E DO TIPO RESULT PARA RESULT<T> (SEGUNDO OPERADOR)

SIMPLIFICA AS **AÇÕES** PARA OS **CLIENTES**, EVITANDO QUE ELES PRECISEM REALIZAR ESSE **TRATAMENTO**

ORIENTAÇÃO A OBJETOS - TRATAMENTO DE FALHAS

```
public override bool CanRegister(Student student)
    => IsOpen() && student.Age <= AgeLimit;
```



```
public override Result CanRegister(Student student)
{
    if (!IsOpen())
        return Result.Failure("Turma não está aberta.");

    if (student.Age > AgeLimit)
        return Result.Failure("Idade superior ao permitido para a turma.");

    return Result.Success();
}
```

USO DE BIBLIOTECAS CONSOLIDADAS

EXISTEM **BIBLIOTECAS CONSOLIDADAS** QUE TRAZEM ESSAS **IMPLEMENTAÇÕES** PRONTAS

- CSharpFunctionalExtensions

ORIENTAÇÃO A OBJETOS - TRATAMENTO DE FALHAS

```
public async Task<Maybe<Professional>> GetByIdAsync(Guid id)
{
    var professional = await _context.Professionals.FirstOrDefaultAsync(p => p.Id == id);
    if (professional is null)
        return Maybe.None;
    return professional;
}
```

Null não é propagado

```
var professional = await _professionalRepository.GetByIdAsync(request.UserId);
if (professional.HasNoValue)
    return Result.Failure("Profissional não encontrado.");
```

Força o cliente a checar se foi possível recuperar o profissional

CAPÍTULO 6

S.O.L.I.D NA PRÁTICA

Elemar

"**UMA CLASSE DEVE TER APENAS UMA RAZÃO PARA MUDAR**"

O CONCEITO CHAVE AQUI É A **RAZÃO PARA MUDAR**

SINGLE RESPONSIBILITY PRINCIPLE (SRP)

```
public class RegistrationService
{
    public Result<Registration> Register(Session session)
    {
        /* code  */
    }

    public IEnumerable<Session> Validate(Session session, Student student)
    {
        /* code  */
    }

    public Result Cancel(Session session, Student student)
    {
        /* code  */
    }
}
```

NÃO É SOBRE "TER UM MÉTODO SÓ", NEM "FAZER POUCO", É **SOBRE QUEM TE FORÇA A MUDAR AQUELE CÓDIGO**.

SRP **NÃO É SOBRE "MICRO-CLASSES POR ESPORTE"**. É SOBRE **ISOLAMENTO DE MUDANÇAS** E **REDUÇÃO DE *BLAST RADIUS***

SINGLE RESPONSIBILITY PRINCIPLE (SRP)

```
public class OrderService
{
    public async Task<Guid> CreateOrder(CreateOrderRequest req)
    {
        // 1) validações
        if (req.Items.Count == 0) throw new Exception("No items");
        if (req.CustomerId == Guid.Empty) throw new Exception("Invalid customer");

        // 2) cálculo
        var total = req.Items.Sum(i => i.Price * i.Quantity);
        if (req.CouponCode == "PROMO10") total *= 0.9m;

        // 3) persistência
        using var conn = new SqlConnection("...");
        await conn.ExecuteAsync("INSERT ...");

        // 4) integração (evento)
        var kafka = new KafkaProducer("...");
        await kafka.PublishAsync("orders", new { /*...*/ });

        // 5) notificação
        var sendGrid = new SendGridClient("...");
        await sendGrid.SendEmailAsync(/*...*/);

        return Guid.NewGuid();
    }
}
```

UMA CLASSE DEVE **SERVIR UM ÚNICO "INTERESSADO"** (NEGÓCIO, COMPLIANCE, INFRA, INTEGRAÇÕES ETC).

SE DUAS ÁREAS DIFERENTES PEDEM MUDANÇAS NA MESMA CLASSE, CRIAMOS UM "**PONTO DE CONFLITO**"

SINGLE RESPONSIBILITY PRINCIPLE (SRP)

```
public sealed class OrderManager(
     OrderRepository repo,
        IOrderEventPublisher publisher,
        IEmailSender email)
{

    public async Task<Guid> CreateOrder(CreateOrderRequest req)
    {
        // responsabilidade: orquestrar (não conter regra)
        var order = Order.Create(req.CustomerId, req.Items, req.CouponCode);

        await repo.Save(order);

        await publisher.PublishOrderCreated(order);

        await email.SendOrderConfirmation(req.CustomerId, order.Id);

        return order.Id;
    }
}
```

SINAIS DE ALERTA:

- CLASSE COM SUFIXO SERVICE/MANAGER/HELPER COM 100+ LINHAS

- MISTURA DE SQL, REGRAS DE VALIDAÇÃO, HTTP, EVENTOS, EMAIL, ETC NO CÓDIGO

- MUDANÇAS RECORRENTES POR TIMES DIFERENTES NO MESMO ARQUIVO

SINGLE RESPONSIBILITY PRINCIPLE (SRP)

FACILITA A **MANUTENIBILIDADE** DO CÓDIGO

CÓDIGO LIMPO E DE FÁCIL ENTENDIMENTO

FAVORECE O **DESENVOLVIMENTO DE TESTES**

REDUZ O **ACOPLAMENTO**

REDUZ A **COMPLEXIDADE**

PROMOVE **MAIOR COESÃO**

OPEN/CLOSED PRINCIPLE (OCP)

"**ENTIDADES DE SOFTWARE (CLASSES, MÓDULOS, FUNÇÕES) DEVEM ESTAR ABERTAS PARA EXTENSÃO E FECHADAS PARA MODIFICAÇÃO**"

ADICIONAR COMPORTAMENTO NOVO SEM FICAR EDITANDO CÓDIGO EXISTENTE (PRINCIPALMENTE O CÓDIGO “ESTÁVEL” E CENTRAL).

OPEN/CLOSED PRINCIPLE (OCP)

```
public class PaymentService
{
    public Task Pay(PaymentRequest request)
    {
        return request.Method switch
        {
            PaymentMethod.CreditCard => PayWithCard(request),
            PaymentMethod.Boleto     => PayWithBoleto(request),
            PaymentMethod.Pix        => PayWithPix(request),
            _ => throw new NotSupportedException()
        };
    }

    private Task PayWithCard(PaymentRequest r) { /*...*/ }
    private Task PayWithBoleto(PaymentRequest r) { /*...*/ }
    private Task PayWithPix(PaymentRequest r) { /*...*/ }
}
```

NO DIA A DIA DE EMPRESA, OCP É UMA **RESPOSTA DIRETA AO "EFEITO DOMINÓ"**: VOCÊ ALTERA UM SWITCH/IF CENTRAL PARA COLOCAR UMA NOVA REGRA.

QUANDO SURGIR UM NOVO CASO, DEVEMOS **ADICIONAR UMA NOVA IMPLEMENTAÇÃO (EXTENSÃO)** EM VEZ DE MEXER NO NÚCLEO.

OPEN/CLOSED PRINCIPLE (OCP)

```
public sealed class CreditCardProcessor : IPaymentProcessor
{
    public PaymentMethod Method => PaymentMethod.CreditCard;

    public Task Process(PaymentRequest request)
    {
        // INTEGRA COM ADQUIRENTE, ANTIFRAUDE ETC.
        return Task.CompletedTask;
    }
}

public sealed class PixProcessor : IPaymentProcessor
{
    public PaymentMethod Method => PaymentMethod.Pix;

    public Task Process(PaymentRequest request)
    {
        // GERA QR CODE, REGISTRA COBRANÇA, WEBHOOK ETC.
        return Task.CompletedTask;
    }
}
```

OPEN/CLOSED PRINCIPLE (OCP)

```
public sealed class PaymentService
{
    private readonly IReadOnlyDictionary<PaymentMethod, IPaymentProcessor> _processors;

    public PaymentService(IEnumerable<IPaymentProcessor> processors)
    {
        _processors = processors.ToDictionary(p => p.Method);
    }

    public Task Pay(PaymentRequest request)
    {
        if (!_processors.TryGetValue(request.Method, out var processor))
            throw new NotSupportedException($"Payment method {request.Method} not supported.");

        return processor.Process(request);
    }
}
```

SE TODA VEZ QUE APARECE UM “CASO NOVO” VOCÊ PRECISA **EDITAR UMA LÓGICA CENTRAL**, **VOCÊ ESTÁ ACUMULANDO DÍVIDA E VIOLANDO OCP**

VANTAGENS DO OPEN-CLOSED PRINCIPLE

FACILITA A **MANUTENIBILIDADE** DO **CÓDIGO**

FAVORECE O **DESENVOLVIMENTO** DE **TESTES**

FAVORECE A **EXTENSIBILIDADE** DO **CÓDIGO**

LISKOV SUBSTITUTION PRINCIPLE (LSP)

SE S É SUBTIPO DE T, ENTÃO OBJETOS DO TIPO T DEVEM PODER SER **SUBSTITUÍDOS POR OBJETOS** DO TIPO S **SEM QUEBRAR A CORREÇÃO DO PROGRAMA**

HERANÇA/IMPLEMENTAÇÃO DE INTERFACE NÃO É SÓ "COMPILAR". **É MANTER O MESMO CONTRATO DE COMPORTAMENTO**

LISKOV SUBSTITUTION PRINCIPLE (LSP)

```
public sealed class S3FileStorage : IFileStorage
{
    public async Task<Uri> Save(string fileName, Stream content)
    {
        if (content.Length > 5_000_000)
            return new Uri("about:blank");

        // ... FAZ UPLOAD
        return new Uri($"https://s3.amazonaws.com/bucket/{fileName}");
    }
}
```

NA PRÁTICA, LSP É SOBRE **CONFIANÇA EM CONTRATOS**.

LISKOV SUBSTITUTION PRINCIPLE (LSP)

```
public class HttpApiClient
{
    public virtual Task<string> Get(string url) => /*...*/;
}

public class CachedHttpApiClient : HttpApiClient
{
    public override Task<string> Get(string url)
    {
        if (url.Contains("realtime"))
            throw new InvalidOperationException("CACHE DESNECESSÁRIO"); // SURPRESA
        return /*...*/;
    }
}
```

PROBLEMA: HERANÇA SÓ PRA REAPROVEITAR CÓDIGO

LISKOV SUBSTITUTION PRINCIPLE (LSP)

```
public class BoletoProcessor : IPaymentProcessor
{
    public Task Process(Payment payment)
    {
        // NA VERDADE, BOLETO NAO "PAGA" AGORA
        // ELE GERA UM CODIGO E AGUARDA CONFIRMACAO
        throw new NotSupportedException("BOLETO NAO EH PROCESSAMENTO IMEDIATO");
    }
}
```

PROBLEMA: FALSA GENERALIZAÇÃO (ABSTRAÇÃO ERRADA)

LISKOV SUBSTITUTION PRINCIPLE (LSP)

```
public class LegacyUserDirectory : IUserDirectory
{
    public Task<User> GetByEmail(string email)
    {
        // QUANDO NAO ACHA, DEVOLVE USUARIO VAZIO (RESULTADO MAIS FRACO)
        return Task.FromResult(new User(Guid.Empty, "unknown@unknown"));
    }
}
```

PROBLEMA: DEVOLVER RESULTADO MAIS "FRACO"

LISKOV SUBSTITUTION PRINCIPLE (LSP)

```
public sealed record SaveFileResult(bool Success, Uri? Uri, string? ErrorCode, string? ErrorMessage);

public interface IFileStorage
{
    Task<SaveFileResult> Save(string fileName, Stream content, CancellationToken ct = default);
}
```

CONTRATO COM RESULTADO RICO

SE O CHAMADOR **PRECISA SABER "QUAL IMPLEMENTAÇÃO É" PRA USAR SEM QUEBRAR**, LSP JÁ FOI VIOLADO

VANTAGENS DO LISKOV SUBSTITUTION PRINCIPLE

SEGURANÇA DE QUE AS CLASSES **BASES** E **DERIVADAS** PODEM SER **INTERCAMBIADAS**

FAVORECE A **EXTENSIBILIDADE** DO CÓDIGO

INTERFACE SEGREGATION PRINCIPLE (ISP)

CLIENTES NÃO DEVEM SER **FORÇADOS A DEPENDER DE MÉTODOS QUE NÃO USAM**

EM **VEZ DE UMA INTERFACE GIGANTE** PARA TUDO, **CRIE INTERFACES MENORES** POR CAPACIDADE/CLIENTE

INTERFACE SEGREGATION PRINCIPLE (ISP)

```
public interface IRepository<T>
{
    // READ
    Task<T?> GetById(Guid id);
    Task<IReadOnlyList<T>> Query(string filter);

    // WRITE
    Task Add(T entity);
    Task Update(T entity);
    Task Delete(Guid id);

    // TRANSACTION
    Task BeginTransaction();
    Task Commit();
    Task Rollback();

    // “EXTRAS”
    Task BulkInsert(IEnumerable<T> entities);
    Task ExecuteRawSql(string sql);
}
```

NA PRÁTICA, ISP APARECE QUANDO UM TIME TENTA PADRONIZAR DEMAIS E **CRIA UM "CONTRATO UNIVERSAL"**.

- QUASE **NINGUÉM USA TUDO**
- **TODOS SÃO IMPACTADOS** QUANDO ALGO MUDA
- IMPLEMENTAÇÕES **FICAM CHEIAS DE "NÃO SUPORTO"**
- TESTES VIRAM UM **INFERNO DE MOCK**

INTERFACE SEGREGATION PRINCIPLE (ISP)

```
public interface IReadRepository<T>
{
    Task<T?> GetById(Guid id);
    Task<IReadOnlyList<T>> Query(string filter);
}

public interface IWriteRepository<T>
{
    Task Add(T entity);
    Task Update(T entity);
    Task Delete(Guid id);
}

public interface IUnitOfWork
{
    Task Begin();
    Task Commit();
    Task Rollback();
}

public interface IBulkInsert<T>
{
    Task BulkInsert(IEnumerable<T> entities);
}

public interface IRawSqlExecutor
{
    Task Execute(string sql);
}
```

VANTAGENS DO INTERFACE SEGREGATION PRINCIPLE

MELHORA A **PRODUTIVIDADE** AO EXIGIR A **IMPLEMENTAÇÃO** APENAS DOS **MÉTODOS NECESSÁRIOS**

REDUZ A **COMPLEXIDADE**

FACILITA A **MANUTENIBILIDADE** DO CÓDIGO

DEPENDENCY INVERSION PRINCIPLE (DIP)

MÓDULOS DE ALTO NÍVEL NÃO DEVEM DEPENDER DE MÓDULOS DE BAIXO NÍVEL. **AMBOS DEVEM DEPENDER DE ABSTRAÇÕES**.

ABSTRAÇÕES NÃO DEVEM DEPENDER DE DETALHES. DETALHES (IMPLEMENTAÇÕES) DEVEM DEPENDER DE ABSTRAÇÕES.

DEPENDENCY INVERSION PRINCIPLE (DIP)

```
public sealed class CreateOrderManager
{
    public async Task<Guid> Execute(CreateOrderRequest req)
    {
        // HTTP DIRETO
        var http = new HttpClient();
        var stockOk = await
http.GetStringAsync($"https://stock/api/check/{req.ProductId}");

        if (stockOk != "OK")
            throw new InvalidOperationException("OUT OF STOCK");

        // BANCO DIRETO
        using var conn = new SqlConnection("Server=...");
        await conn.ExecuteAsync("INSERT INTO Orders ...");

        // KAFKA DIRETO
        var producer = new KafkaProducer("broker...");
        await producer.PublishAsync("orders", new { /*...*/ });

        return Guid.NewGuid();
    }
}
```

REGRA DE NEGÓCIO NÃO DEVE CONHECER FRAMEWORK, BANCO, HTTP, SDK DE TERCEIRO.

ISSO FICA “POR FORA”, **ATRÁS DE CONTRATOS**.

DEPENDENCY INVERSION PRINCIPLE (DIP)

```
public sealed class CreateOrderUseCase(IStockGateway stock, IOrderRepository repo, IEventPublisher publisher)
{
    public async Task<Guid> Execute(CreateOrderRequest req, CancellationToken ct)
    {
        if (!await stock.HasStock(req.ProductId, req.Quantity, ct))
            throw new InvalidOperationException("OUT OF STOCK");

        var order = Order.Create(req.CustomerId, req.ProductId, req.Quantity);

        await repo.Save(order, ct);
        await publisher.PublishOrderCreated(order, ct);

        return order.Id;
    }
}
```

DEPENDENCY INVERSION PRINCIPLE (DIP)

```
builder.Services.AddScoped<CreateOrderUseCase>();
builder.Services.AddScoped<IOrderRepository, SqlOrderRepository>();
builder.Services.AddScoped<IEventPublisher, KafkaEventPublisher>();

builder.Services.AddHttpClient<IStockGateway, HttpStockGateway>(client =>
{
    client.BaseAddress = new Uri("https://stock");
});
```

COMPOSITION ROOT QUE "CONECTA TUDO"

VANTAGENS DO DEPENDENCY INVERSION PRINCIPLE

DIMINUI O **ACOPLAMENTO**

FAVORECE A **ESCRITA DE TESTES**

AUMENTA A **FLEXIBILIDADE** DO CÓDIGO

QUAL SUA OPINIÃO

PARA CADA **CLASSE** DE **SERVIÇO**, DEVEMOS TER UMA **INTERFACE**?

QUAL A MOTIVAÇÃO?

```
public class EmailService
{
    public async Task SendEmailAsync(string to, string subject, string body)
    {
        // Faz o envio do e-mail
    }
}


public interface IEmailService
{
    Task SendEmailAsync(string to, string subject, string body);
}
```

UMA **INTERFACE** QUE APENAS REPLICA O **CONTRATO** DA CLASSE AGREGA **POUCO VALOR**.
SEU VERDADEIRO PROPÓSITO BRILHA AO **VIABILIZAR MÚLTIPLOS COMPORTAMENTOS**.

CLUBE DE ESTUDOS
BY ELEMAR JR

C#
DO JEITO CERTO

Elemar